Advanced Programming Techniques (B9CY108)
CA-2 Report

Student Names and Numbers as per student card(s):

Arslan Arshad (**20093854**)
Perk Ramchandani (**20095995**)
Sanskar Landge (**20096869**)

Program: MSc in Cybersecurity
Lecturer Name: Ms. Swati Dongre
Module/Subject Title: Advanced Programming Techniques
Assignment Title: **Secure Onboarding Gateway-Automated Attribute-Based Access Control (CA2-Report)**

Deliverables:

- GitHub repository: https://github.com/PerkxCode/CA2_Advanced-Programming/tree/main/20095995-CA2-ABAC
- Demo video: https://drive.google.com/file/d/1FFmvvhR3RpyZN8UXYwmqkIqMymxHoeDO/view?usp=sharing

By submitting this assignment, we are confirming that:

1. This assignment is all our own work.
2. Any sources used have been referenced.
3. We have followed the Generative AI instructions/ scale set out in the Assignment Brief.
4. We have read the College rules regarding academic integrity in the QAH Part B Section 3, and the Generative AI Guidelines, and understand that penalties will be applied accordingly if work is found not to be my/our own.

We understand that all work uploaded is submitted via Original, whereby a text-matching report will show any similarities with other texts.

Secure Onboarding Gateway: Automated Attribute-Based Access Control (ABAC)

Advanced Programming Techniques – CA2 Project Report

1. Problem Domain and Project Scope

The selected problem domain is secure employee onboarding with automated access control and security monitoring. The project is titled Secure Onboarding Gateway: Automated Attribute-Based Access Control, and its aim is to show how onboarding can be treated as both an IAM process and a security monitoring process. In many organisations, account creation is handled quickly, but the first stage of access management is often spread across emails, spreadsheets, and manual approval actions. That creates risk because approval, password delivery, login validation, and alert monitoring are not always joined together. This project was designed to centralise those tasks inside one information system.

The system is therefore not only a registration portal. It supports employee onboarding, admin approval, first-login password change, secure sign-in, activity logging, account lockout, admin unlock control, and security alert monitoring. This makes the project suitable for the chosen title because it uses user attributes and account state together to influence access behaviour. It also reflects the group's combined IAM and SOC interests. Even though the system is an academic prototype, it addresses a real security problem: new employees should not receive full operational access until identity details are captured, reviewed, approved, and monitored.

2. Requirements Elicitation and Software Development Life Cycle

The main functional requirements were derived from the project theme and from the assignment brief. The system had to support employee registration, admin approval and decline, secure login, password update on first login, user and admin dashboards, employee dossier viewing, activity logging, account lockout after repeated failed attempts, unlock actions by the admin, and monitoring alerts through a separate service. Because the project also needed to demonstrate a complete information system, it had to include a frontend, backend logic, persistent storage, execution screenshots, a demonstration video, and a GitHub repository.

The non-functional requirements were also important. The application needed to be simple enough to complete within the academic timeline, but still secure, organised, readable, and suitable for demonstration. It also needed to show clear evidence of architecture, data handling, security controls, and testing. For this reason, the project followed an iterative development approach rather than a rigid waterfall approach. A strict waterfall model would have made it difficult to adjust the frontend, backend, and monitoring logic as new issues appeared. Instead, the project was built in stages. First, the database structure and models were created. Then the onboarding and login

flow was implemented. After that, the admin approval and password management features were added. Finally, the activity logging, lockout logic, and TCP monitoring component were integrated. This iterative process reduced risk and made debugging easier because the team could test each layer while building the next one.

3. Analysis, Design and Layered Architecture

The project uses layered architecture, which was selected because it clearly separates the user interface, the business rules, and the data handling layer. This directly satisfies the brief requirement to justify an architectural pattern for the backend, frontend, and business logic components. The layered design also improves maintainability because changes to one area of the system are less likely to break another area.

The presentation layer is implemented with Flask templates and HTML pages. The uploaded templates show a shared base layout, a signup page for employee onboarding, a secure login page, a password change page, a user dashboard, an admin dashboard, and an employee dossier page. The base template provides the common navigation and flash-message area, while the CSS file provides the visual theme with cards, grids, alerts, badges, forms, and tables. This means the frontend is not just decorative. It supports real workflow steps such as entering onboarding data, signing in, reviewing pending approvals, viewing active employees, examining activity logs, and performing unlock or approval actions.

The business logic layer is implemented mainly in the Flask application and in the separate monitoring server. The Flask app handles the onboarding route, login checks, logout, admin dashboard logic, approval and decline actions, user dashboard access, and forced password change on first login. The monitoring server listens on localhost port 9999 and receives JSON alerts through a socket connection. That gives the project a second operational component and supports the SOC side of the idea.

The data layer is implemented through Flask-SQLAlchemy over SQLite. The User model stores identity, organisational, and security-related attributes including name, email, mobile, department, access requirement, location, job title, payroll, role, approval state, failed attempts, lock status, and force logout state. The ActivityLog model stores user activity, device, location, and timestamp, and links each event back to a user. This layered arrangement is well suited to a secure onboarding system because the frontend captures user data, the business layer enforces policy and monitoring rules, and the database preserves auditable records.

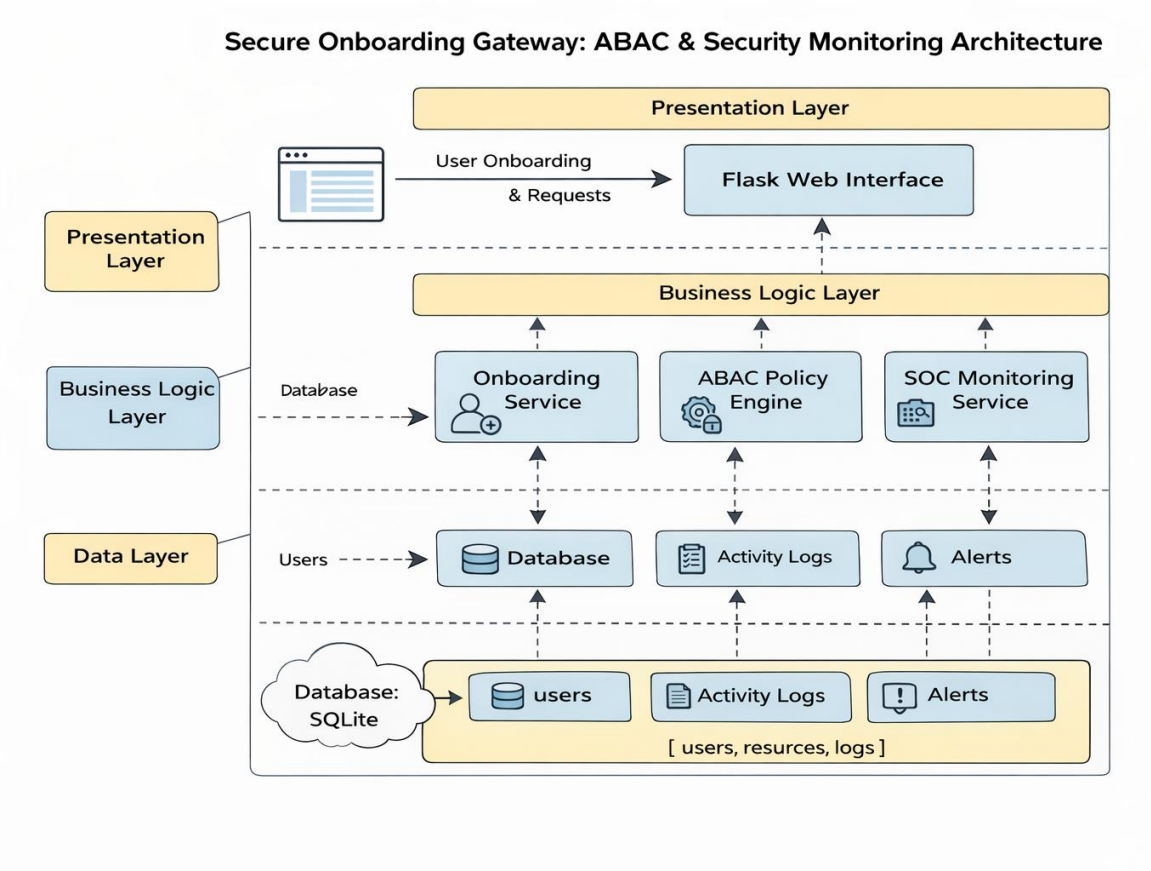

Figure 1: Layered architecture of the Secure Onboarding Gateway with ABAC

4. Data Structures, Algorithms and Rationale

The project uses the relational database as its main data structure because onboarding and monitoring both depend on well-structured records. User data is naturally represented as rows with defined attributes, while activity events are better stored as a separate log table. A one-to-many relationship between users and activity logs is therefore an appropriate design. It avoids duplication of employee details while still making it possible to review user-specific or global activity histories.

At the application level, JSON is used for alert messaging between the Flask application and the monitoring server. This is a good choice because the alert data is small, structured, and easy to extend. When an alert is generated, a dictionary is assembled in Python and then serialized into JSON before being sent over the socket. This makes the monitoring payload readable and flexible.

The core decision logic of the system is rule-based rather than probabilistic. During login, the application checks whether the user exists, whether the account is approved, whether the account is locked, whether the submitted location matches the registered location, and whether the password hash matches the entered password. If any of these checks fail, the system applies the correct branch of logic. Repeated password failures increase the failed-attempt counter. At a warning threshold the user is informed, a log is written, and an email can be sent. At the lockout threshold the account is disabled, and the monitoring server is notified. This algorithmic approach is suitable because it is deterministic, easy to justify, and easy to test in an academic setting.

5. Technologies, Libraries and External Services

Python was selected because it satisfies the assignment requirement and supports rapid implementation of a full-stack academic system. Flask was used because it is lightweight and suitable for a layered design with routing and template rendering. Flask-SQLAlchemy simplified database work by allowing the project to define models and relationships in Python rather than writing low-level database code for each operation. SQLite was chosen because it is simple, portable, and easy to inspect in DBeaver, which made both testing and screenshot collection easier.

Werkzeug security functions were used for password hashing and verification, which is a much better practice than storing plain passwords. The project also uses `socket` and `json` for the monitoring channel, `smtplib` and MIME components for email notifications, `secrets` for temporary password generation, and `pytz` for consistent Ireland-based timestamps. The CSS file defines a straightforward dashboard theme and shows that the interface was intentionally structured rather than left in default browser formatting.

Although the brief mentions APIs, this project did not depend on an external REST API because the main business case is secure onboarding and internal monitoring. Instead of adding a weakly justified third-party API only to satisfy terminology, the project uses two meaningful integrations: SMTP-based email delivery and an internal TCP monitoring service. These choices are directly relevant to the problem domain and easier to defend in the report.

6. Security Measures and Mitigation Strategies

Security was central to the design. First, passwords are hashed using Werkzeug before storage, which protects user credentials if the database is exposed. Second, newly approved users receive a temporary password and must change it on first login, reducing the time that system-generated passwords remain active. Third, the system blocks login for users whose accounts are still pending admin approval, which prevents new users from bypassing the review process.

A fourth control is failed-attempt tracking. Each incorrect password attempt increases a counter. At three failures the user receives a warning and the event is logged. At the lockout threshold the account is marked as locked, the time is recorded, and the monitoring server receives an alert. A fifth control is location checking. The login route compares the submitted department location against the registered location and blocks mismatches. A sixth control is visibility through logging. Important actions such as onboarding completion, password change, account approval, lockout, and unlock are stored in the ActivityLog table. This creates an audit trail that can be reviewed by the admin.

The monitoring server strengthens the SOC aspect of the project. It receives alert payloads containing username, email, location, failed attempts, account lock state, and timestamps. This means suspicious behavior is not hidden inside the web application only. It is surfaced to a separate monitoring component, which reflects a practical SOC mindset even though the project remains intentionally lightweight.

7. System Implementation and Screenshots

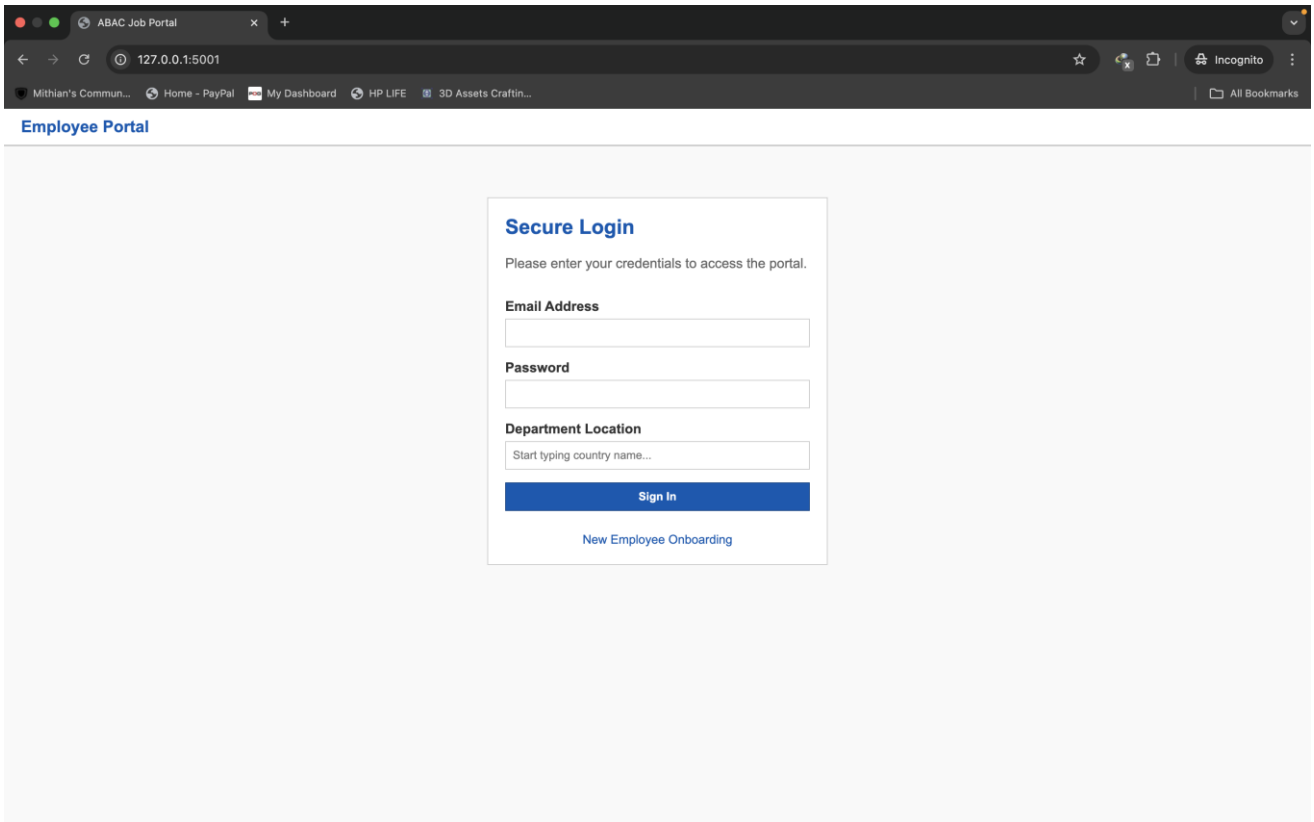

Figure 2: Home page of the Secure Onboarding Gateway system

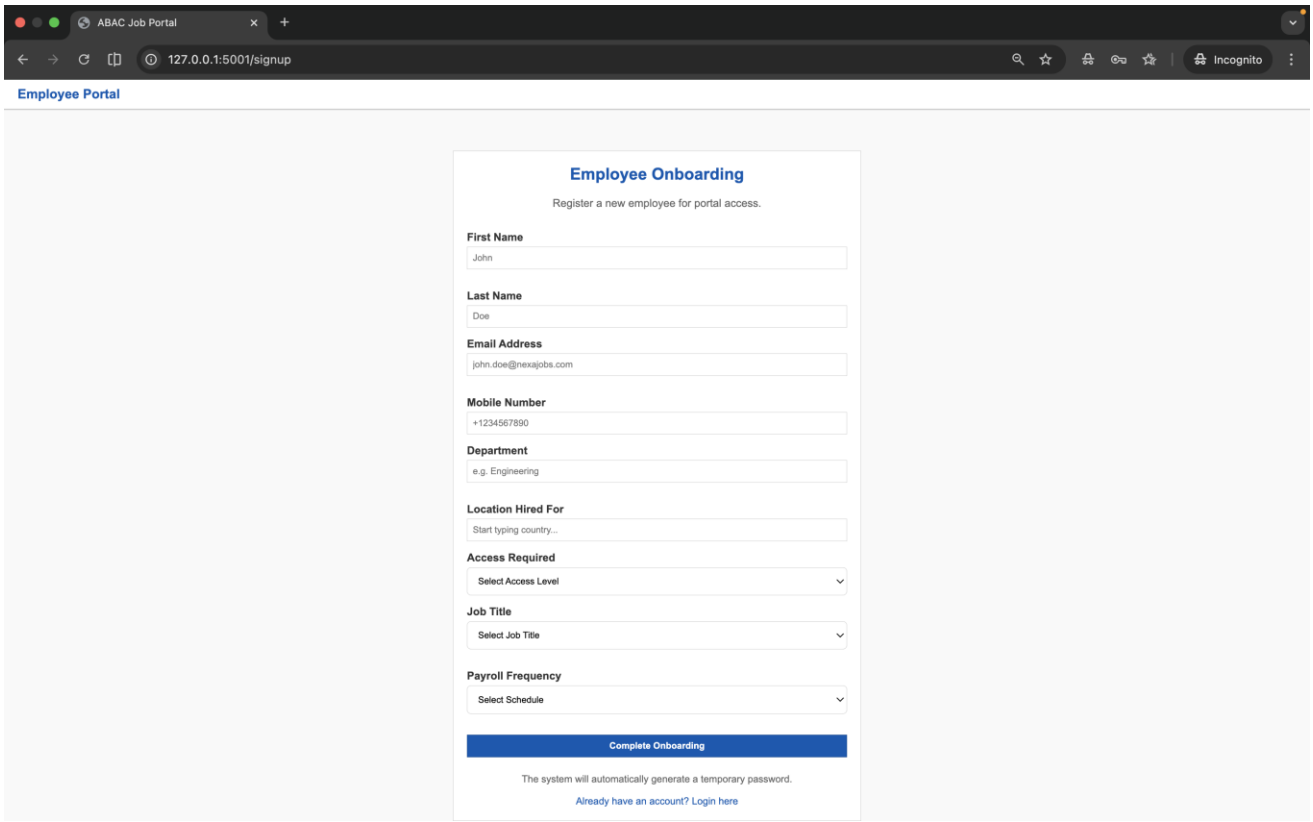

Figure 3: User onboarding form capturing attributes for ABAC evaluation

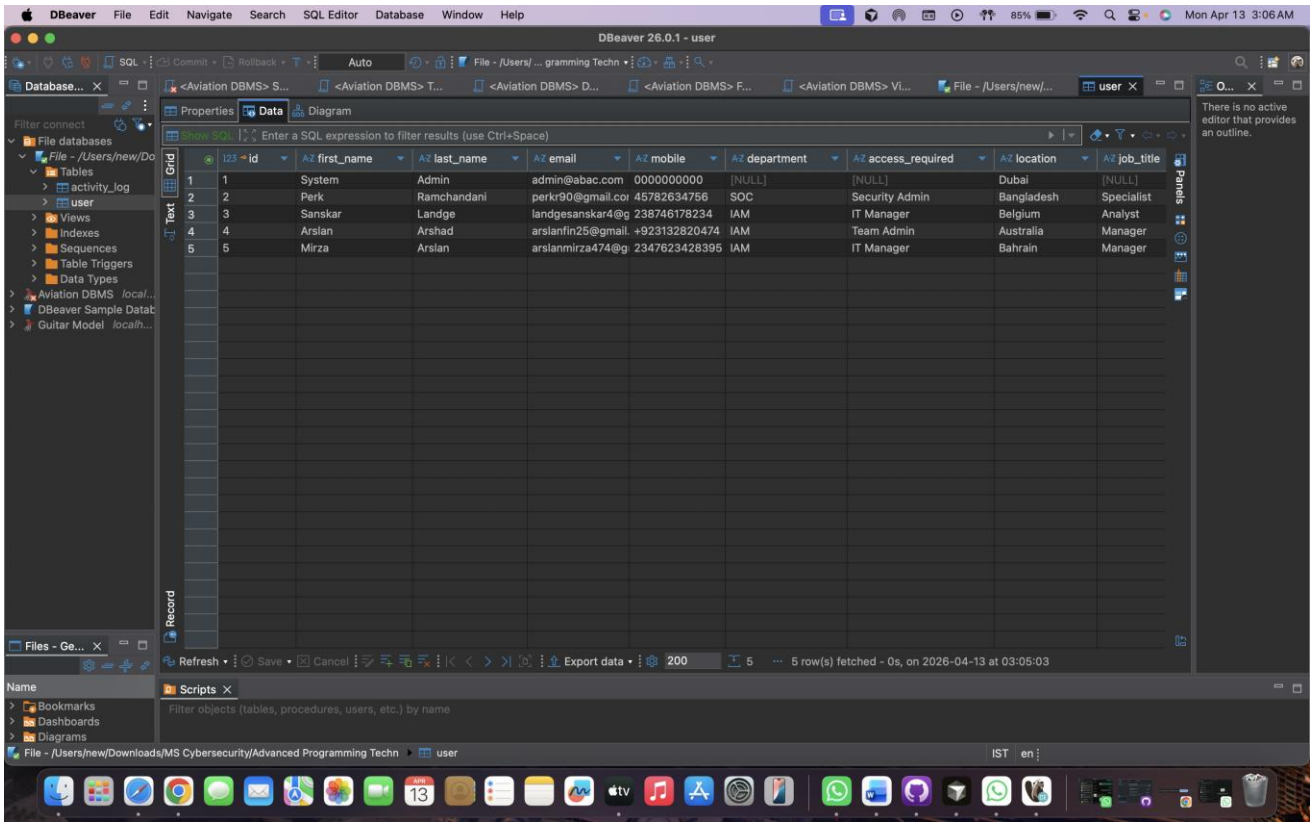

Figure 4: User data stored in SQLite database viewed using DBeaver

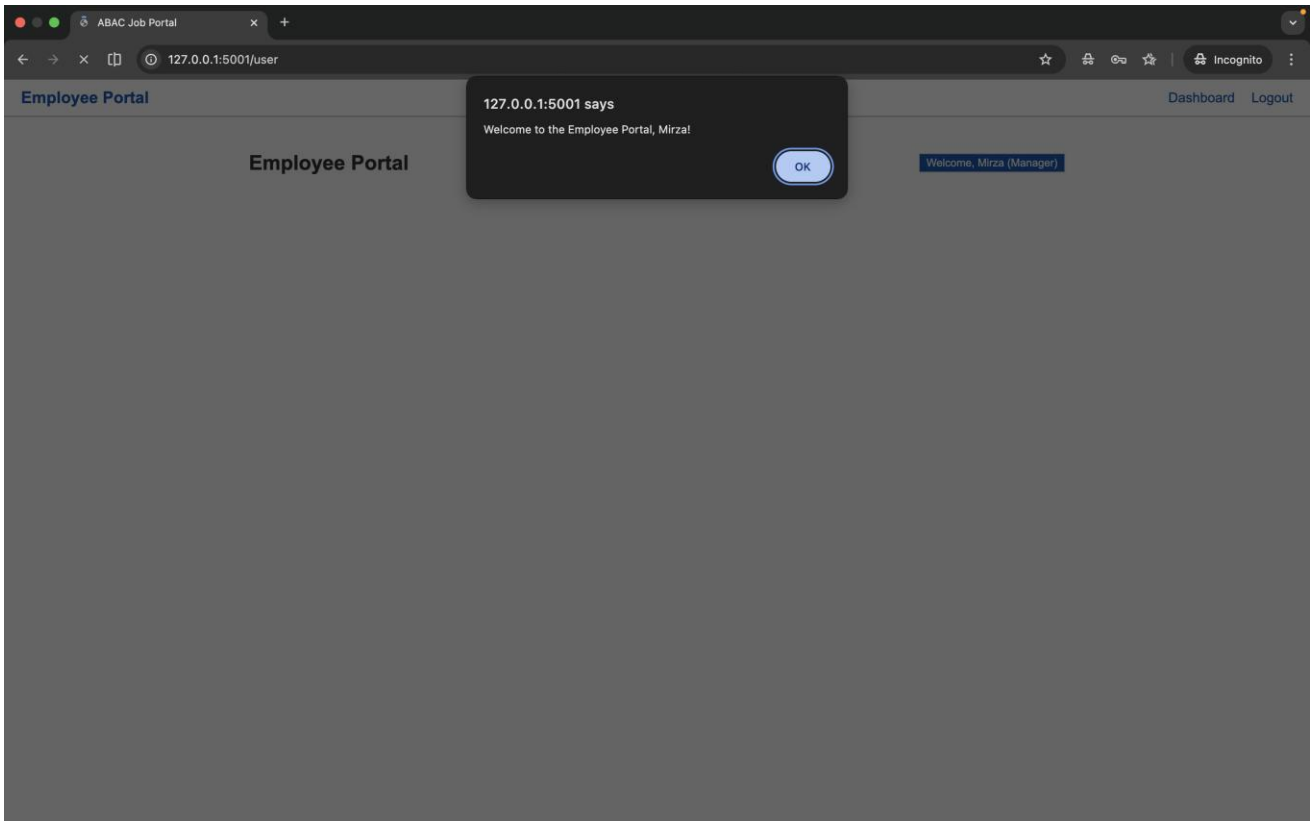

Figure 5: ABAC access decision based on user attributes

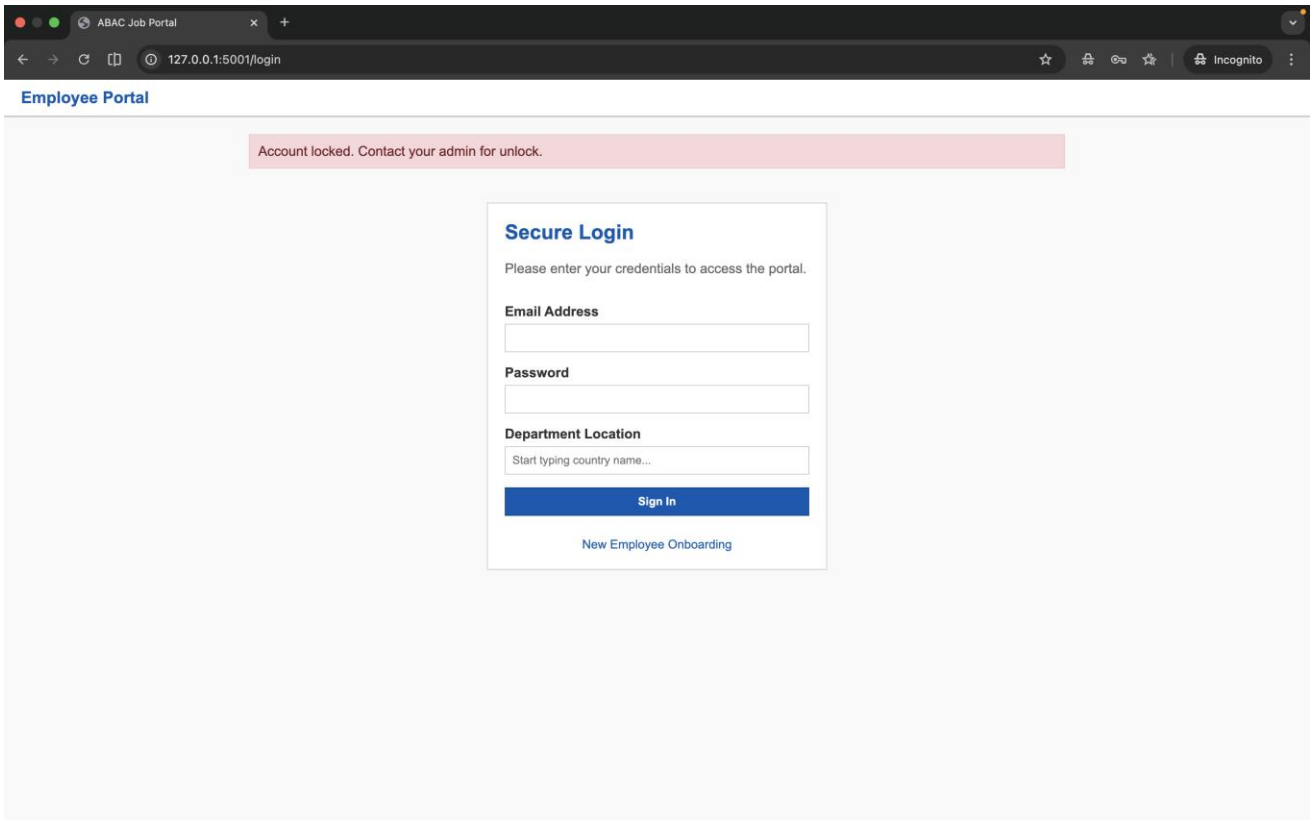

Figure 6: Detection of suspicious activity leading to account lock

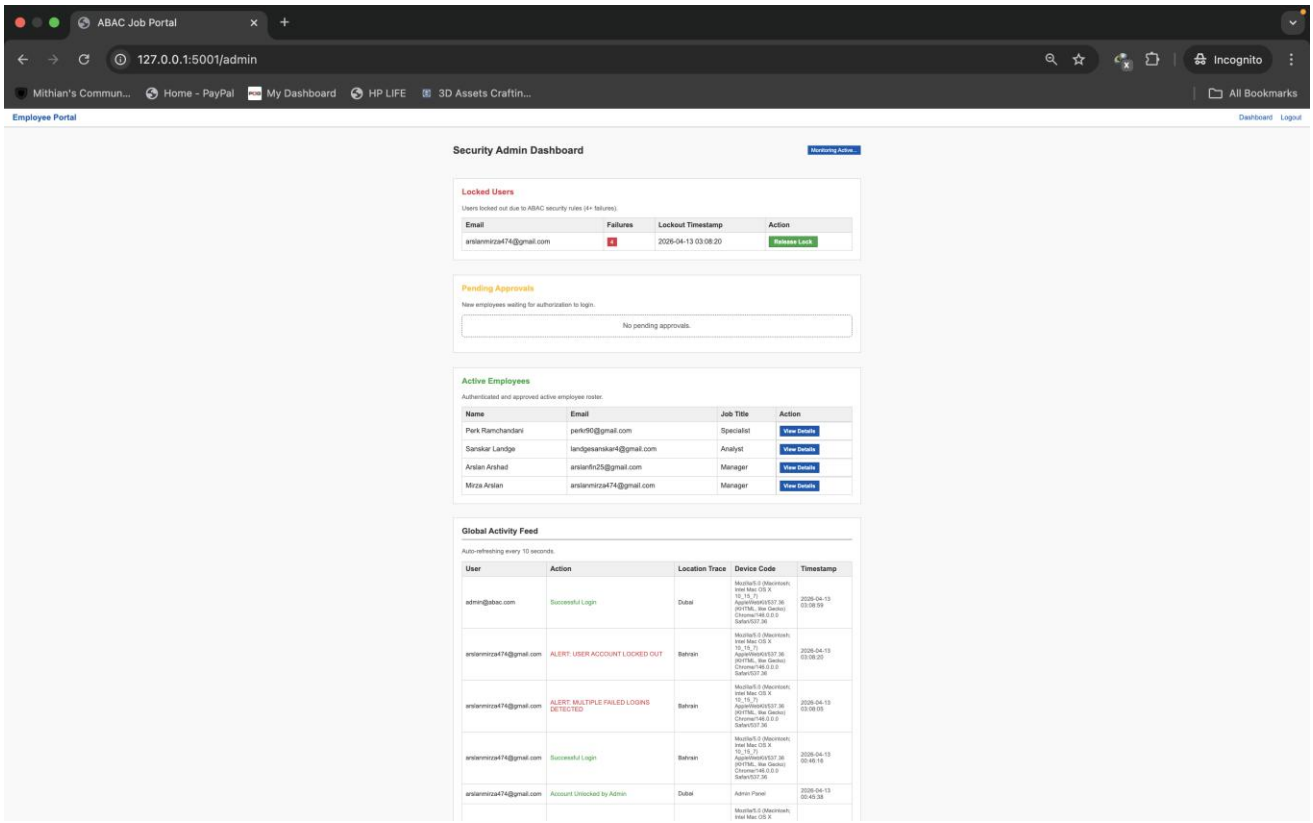

Figure 7: Admin dashboard displaying system activity and monitoring information

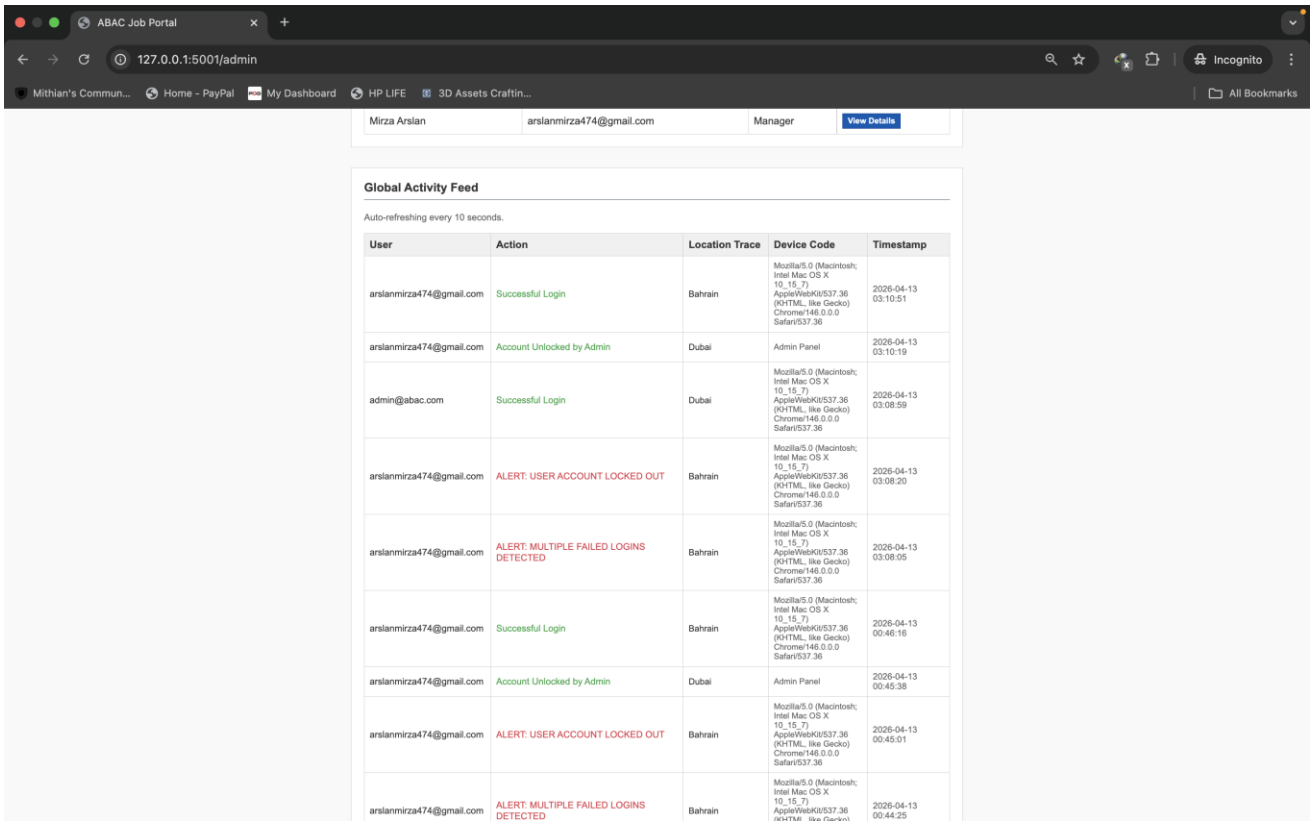

Figure 8: SOC monitoring logs capturing system events and user actions

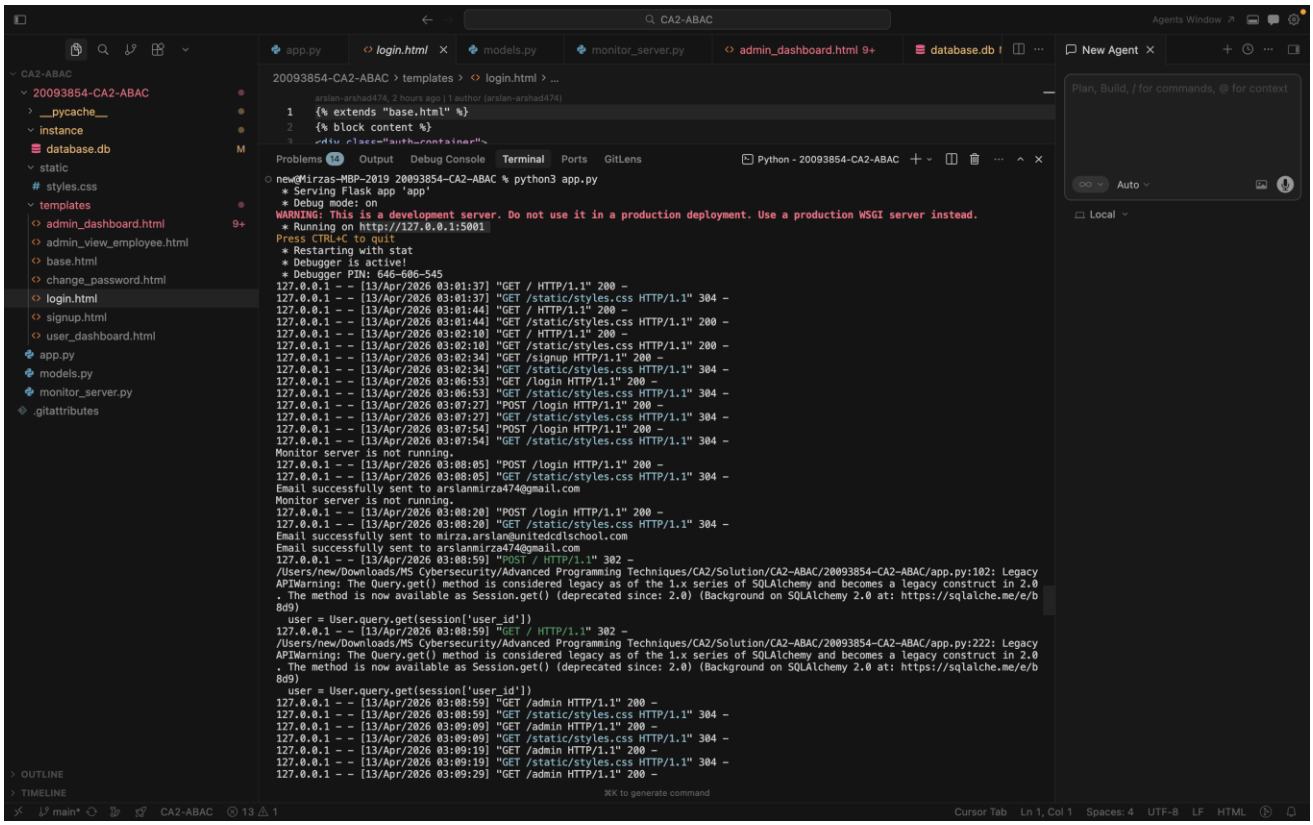

Figure 9: Activity logs on and API calls in real-time on console

8. Testing Strategy, Execution Evidence and Evaluation

Testing was carried out through staged functional testing and integration testing. The team tested successful signup, duplicate email handling, pending approval login, admin approval, temporary password generation, first-login password update, valid login, invalid password behaviour, repeated failures, account lockout, admin unlock, and viewing the activity feeds. The SQLite database was reviewed in DBeaver to confirm that user records and activity logs were written correctly. The monitoring server was also tested by forcing failed login scenarios and checking whether the expected alert messages were received and printed.

The system produced the expected behaviour across the main workflow. A new employee can register, wait for approval, receive a temporary password, change it at first login, and then access the user dashboard. The admin can review pending onboarding requests, approve or decline them, unlock locked accounts, and inspect global activity. This provides solid evidence that the project is more than a simple login page.

The project demonstration video is approximately 18 minutes long, which is longer than the 8-to-10-minute suggestion in the brief. The main reason is that the application contains several connected modules, and a shorter recording would have required skipping important evidence. The team chose to demonstrate the onboarding form, approval cycle, secure login controls, lockout behaviour, database impact, and monitoring output in one continuous walkthrough so that the

system design decisions could be understood properly. In that sense, the longer demo reflects feature breadth rather than poor planning.

9. GitHub Repository and Conclusion

- GitHub repository: https://github.com/PerkxCode/CA2_Advanced-Programming/tree/main/20095995-CA2-ABAC
- Demo video: https://drive.google.com/file/d/1FFmvvhR3RpyZN8UXYwmqkIqMymxHoeDO/view?usp=sharing

In conclusion, the Secure Onboarding Gateway is a well-justified project for the selected domain. It addresses the real problem of insecure or weakly monitored onboarding by combining employee registration, administrative review, ABAC-style access conditions, logging, and alerting in one layered Python system. The project is stronger than a simple employee portal because it links frontend workflow, backend rules, persistent storage, and a monitoring component. It also successfully reflects the combined IAM and SOC focus of the group. Overall, the final implementation meets the assignment requirements well and provides a practical demonstration of secure onboarding as an information system.

References

1. Flask (n.d.) Flask documentation. Available at: https://flask.palletsprojects.com/ (Accessed: 10 April 2026).
2. Flask-SQLAlchemy (n.d.) Flask-SQLAlchemy documentation. Available at: https://flask-sqlalchemy.palletsprojects.com/ (Accessed: 10 April 2026).
3. OWASP (n.d.) Authentication Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Accessed: 10 April 2026).
4. Python Software Foundation (n.d.) json — JSON encoder and decoder. Available at: https://docs.python.org/3/library/json.html (Accessed: 11 April 2026).
5. Python Software Foundation (n.d.) smtplib — SMTP protocol client. Available at: https://docs.python.org/3/library/smtplib.html (Accessed: 11 April 2026).
6. Python Software Foundation (n.d.) socket — Low-level networking interface. Available at: https://docs.python.org/3/library/socket.html (Accessed: 11 April 2026).
7. SQLAlchemy (n.d.) SQLAlchemy documentation. Available at: https://docs.sqlalchemy.org/ (Accessed: 10 April 2026).
8. Werkzeug (n.d.) Utilities for password hashing. Available at: https://werkzeug.palletsprojects.com/ (Accessed: 11 April 2026).